

Compiler-3 보고서

I. 프로그램 수행 도중 발생한 오류 및 해결 방법

① `getPointerFieldIdentifier`, `set_expression` 함수에서 닫는 괄호를 놓쳐 오류가 발생하였다.(오타)

- 해결 방법: 적절한 위치에 닫는 괄호를 추가하였다.

② `sem_arg_expr_list`에서 `llink`가 아닌 `link`를 적어 오류가 발생하였다.(오타)

- 해결 방법: 오타를 수정하였다.

③ `N_EXP_ARRAY`에서 `t`가 `usual_binary_conversion` 할 때 `t`가 선언되지 않고 처음 사용되어서 오류가 발생하였다.

- 해결 방법: 해당 페이지에 `A_TYPE *t`를 통해 선언을 했다.

④ `main` 함수에서 `semantic_err`를 찾을 수 없다는 `error`가 발생하였다.

- 해결 방법: `extern` 선언을 해서 외부에서 해당 값을 가져올 수 있게 하였다.

⑤ `sem_func.c`와 `func.c`에 `isPointerOrArrayType` 함수가 중복 선언되어 오류가 발생하였다.

- 해결 방법: `semfunc.c`에 있는 함수를 주석처리 하였다.

*아직 해결하지 못한 부분

1. 프로그램이 리터럴 처리를 제대로 하지 못하는 문제
2. 함수와 배열과 같은 포인터를 처리하지 못하는 문제
3. 구조체 자기 참조 문제

II. Test Case

1.

[입력값]

```
~$ cat ex.c
int hamsoo(){
    int *p=3;
    char a = !(p);
}
```

[출력값]

```
N_PROGRAM (0,12)
| (ID="hamsoo") TYPE:e2c2b980 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | | TYPE
| | | | | (int)
| | | | BODY
| | | | | N_STMT_COMPOUND (0,8)
| | | | | | (ID="p") TYPE:e2c2ba00 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | TYPE
| | | | | | | POINTER
| | | | | | | | ELEMENT_TYPE
| | | | | | | | (int)
| | | | | | | INIT
| | | | | | | | N_INIT_LIST_ONE (0,0)
| | | | | | | | | N_EXP_INT_CONST (0,0)
| | | | | | | | | INT=3
| | | | | | (ID="a") TYPE:e2c26410 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=16
| | | | | | TYPE
| | | | | | (char 1)
| | | | | | INIT
| | | | | | | N_INIT_LIST_ONE (0,0)
| | | | | | | | N_EXP_NOT (0,0)
| | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | (ID="p") TYPE:e2c2ba00 KIND:VAR SPEC=AUTO LEV=1 VAL=0
ADDR=12
| | | | | | N_STMT_LIST_NIL (0,0)
~$ gcc ex.c
ex.c: In function 'hamsoo':
ex.c:2:16: warning: initialization of 'int *' from 'int' makes pointer from integer without a cast [-Wint-conversion]
   2 |     int *p=3;
     |
```

[설명]

시멘틱 트리를 확인했을 때 level과 주소, type, kind가 잘 들어있는 것을 확인할 수 있다. 함수의 주소는 0으로 처리되어 있고 처음 등장한 p, a 순으로 4바이트 크기를 가져 각각 12, 16 주소를 갖는 것이 확인된다. 그리고 arithmetic 타입이 아닌 값은 예외가 발생해야 정상인데 문제가 발생하지 않고 있다. 구조체, 공용체, 포인터 모두 제대로 예외 메시지가 출력되지 않는 상태이다. gcc의 결과는 포인터 타입에 int_type을 casting이 발생해 경고 메시지가 나오고 있다.

2.

[입력값]

```
enum Day { first, second=2, third=8 };

int main(){
    int a=second++;
}
```

[출력값]

```
(ID="") TYPE:d83a1960 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=12
| TYPE
| | ENUM
| | | ENUMERATORS
| | | | (ID="first") TYPE:0 KIND:ENUM_LITERAL SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | (ID="second") TYPE:0 KIND:ENUM_LITERAL SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | INIT
| | | | | INT=2
| | | | (ID="third") TYPE:0 KIND:ENUM_LITERAL SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | INIT
| | | | | INT=8
(ID="main") TYPE:d83a1c70 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| TYPE
| | FUNCTION
| | | PARAMETER
| | | TYPE
| | | | (int)
| | | BODY
| | | | N_STMT_COMPOUND (0,4)
| | | | | (ID="a") TYPE:d839c2f0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | TYPE
| | | | | | (int)
| | | | | INIT
| | | | | | N_INIT_LIST_ONE (0,0)
| | | | | | | N_EXP_POST_INC (0,0)
| | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | (ID="second") TYPE:0 KIND:ENUM_LITERAL SPEC=NULL LEV=0
| | | | | | | | VAL=0 ADDR=0
| | | | | | | | N_STMT_LIST_NIL (0,0)
:~$ gcc ex.c
x.c: In function 'main':
x.c:4:21: error: lvalue required as increment operand
  4 |     int a=second++;
    |                   ^~
```

[설명]

열거형이기 때문에 addr 부분이 모두 0으로 되어 있는 것을 확인할 수 있다. 그리고 제작한 컴파일러에서는 열거형 안의 값들이 리터럴 상수로 저장되어 있고, lvalue가 false이기 때문에 후위 증가 수식이 가능하다. 반면 gcc에서는 second가 lvalue 값으로, 후위 증가 수식이 가능하지 않다.

3.

[입력]

```
int main(){
    float x;
    x();
}
```

[출력]

```
**** semantic error at line 3: illegal type in function call expression
: $ gcc ex.c
ex.c: In function 'main':
ex.c:3:9: error: called object 'x' is not a function or function pointer
   3 |     x();
     |     ^
```

[설명]

함수를 호출하게 되면 usual_unary_conversion에 의해 포인터로 바뀌게 되는데, x는 float 형의 scalar 타입이기 때문에 usual_unary_conversion을 하면 변환이 이루어지지 않아 그대로 float 타입이다. 따라서 함수 호출 형태를 사용하면 포인터 타입이 아니고 함수 타입도 아니어서 오류가 발생하게 된다. 또한, gcc 역시도 동일한 원인의 error message를 출력한다.

4.

[입력]

```
int main(){
    char ch = 1+2;
    int a[3];
    char chh = ch + a;
    int b = a + a;
}
```

[출력]

```
ex.c: In function 'main':
ex.c:4:20: warning: initialization of 'char' from 'int *' makes integer from pointer without a cast [-Wint-conversion]
   4 |     char chh = ch + a;
     |                  ^~
ex.c:5:19: error: invalid operands to binary + (have 'int *' and 'int *')
   5 |     int b = a + a;
     |             ^ ~
     |             | |
     |             | int *
     |             int *
```

[설명]

제작한 컴파일러에서는 수식의 연산에서 오류가 발생하지 않고 있다. gcc의 결과와 같이 배열 a는 usual_unary_conversion에 의해 int형 포인터가 돼서 '포인터 + 포인터' 연산이 이루어

어질 때 error message가 발생해야 정상이다. 선택스 트리의 주소, kind, type은 정상이다.

5.

[입력]

```
int main(){
    char ch[10];
    int a=2;
    int b=2;

    char result1 = a && b;
    int result2 = ch && b;
    float result3 = a || b;
    void result4 = b || ch;
}
```

[출력]

```
| | | | | (ID="ch") TYPE:6bd9aa90 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | TYPE
| | | | | ARRAY
| | | | | INDEX
| | | | | INT=10
| | | | | ELEMENT_TYPE
| | | | | (char 1)
| | | | | (ID="a") TYPE:6bd952f0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=24

: $ gcc ex.c
ex.c: In function 'main':
ex.c:9:14: error: variable or field 'result4' declared void
  9 |         void result4 = b || ch;
    |
```

[설명]

시멘틱 트리를 통해 ch가 char 타입의 10byte여도 4의 배수의 align이 적용되는 것을 확인할 수 있다. ch 다음 a의 주소가 ch 주소 + 12인 것을 알 수 있다. ch는 배열 타입으로, usual_unary_conversion에 의해 char 타입의 포인터가 된다. 따라서, scalar 타입이므로, 정상 처리된다. 또한, 모든 void는 모든 타입이 변환할 수 있어 error message가 출력되지 않는다. 하지만 gcc는 void에 값을 넣는 것을 허용하지 않는다.

6.

[입력]

```
union { int a; float b; char c[22]; }x;

int main(){
    int a = x++;
    int b = !x;
    int c = --x;
    int d = x && x;
    int e = x || x;
}
```

[출력]

```
ex.c: In function 'main':
ex.c:4:18: error: wrong type argument to increment
   4 |         int a = x++;
     |                   ^
ex.c:5:17: error: wrong type argument to unary exclamation mark
   5 |         int b = !x;
     |                   ^
ex.c:6:17: error: wrong type argument to decrement
   6 |         int c = --x;
     |                   ^
ex.c:7:17: error: used union type value where scalar is required
   7 |         int d = x && x;
     |                   ^
ex.c:8:17: error: used union type value where scalar is required
   8 |         int e = x || x;
     |                   ^
```

[설명]

union은 scalar 타입이 아니기 때문에 POST_INC, NOT, PRE_DEC, AND, OR가 연산이 되지 않는게 정상이다. 하지만 시멘틱 트리가 주소, kind, type, 나열된 순서에 맞게 출력되는 것을 확인할 수 있다. 반면, gcc에서는 해당 연산들에 적절하지 않은 타입임을 error message로 보여주고 있다.

7.

[입력]

```
int main(){
    int a = 3;
    int b = 5;
    float c = 7;
    char d = 2;
    enum { z, y, x } en;

    int hamsoo();

    int result = a * b;
    result = y * d;
    result = x / z;
    result = c / d;
    result = hamsoo / a;
}
```

[출력]

```
--- warning at line 13:incompatible types in binary expression
--- warning at line 13:incompatible types in assignment expression
**** semantic error at line 14: arith type expression required in binary operation
세그멘테이션 오류 (코어 덤프됨)

$ gcc ex.c

ex.c: In function 'main':
ex.c:12:20: warning: division by zero [-Wdiv-by-zero]
   12 |         result = x / z;
      |                     ^
ex.c:14:25: error: invalid operands to binary / (have 'int (*)()' and 'int')
   14 |         result = hamsoo / a;
      |                     ^
      |                     |
      |                     |
      |                     int (*)()
```

[설명]

제작한 컴파일러에서는 양쪽 모두 arithmetic인 경우(int, float, char, enum)는 정상적으로 출력되는 것이 확인된다. 또한, 13번째 줄에서 float 타입과 char 타입 사이 DIV 연산을 하는데, float이 integral 타입이 아니어서 isIntegralType 검사를 할 때 warning message가 출력된다. 14번째 줄에서는 해당 연산에 맞지 않는 타입이라는 error message를 출력한다. 하지만 여전히 리터럴과 함수를 사용하면 segmentation fault가 발생한다. en의 타입이 T_ENUM, hamsoo의 type이 T_FUNC인 것은 gdb를 통해 확인했으나 segmentation fault가 지속적으로 나타난다. gcc에서는 enum의 첫 변수인 z로 인해서 division by zero warning message를 출력한다. 또한, 함수(포인터)는 해당 연산(MUL, DIV)에 사용할 수 없다는 error message를 보여준다. 나머지는 연산들은 이상 없다.

8.

[입력]

```
int a[5+2.5];
```

[출력]

```
**** semantic error at line 1: illegal array size or type
**** semantic error at line 1: illegal array size or empty array
~$ gcc ex.c
ex.c:1:5: error: size of array 'a' has non-integer type
  1 | int a[5+2.5];
    |           ^
```

[설명]

왼쪽 수식인 a는 배열 타입이 맞고, 오른쪽 수식은 integral(int) 타입이 아니어서 비정상적인 타입이며, 제대로된 계산이 되지 않아 배열의 크기와 관련된 error message도 같이 출력된다. gcc 역시도 a의 크기가 integer 타입이 아니어서 error message를 출력하고 있다.

9.

[입력]

```
int main(){
    int a;
    int result = *a;

    int b;
    result = *b;
}
```

[출력]

```
**** semantic error at line 6: pointer type expr required in pointer operation
세그멘테이션 오류 (코어 덤프됨)
~$ gcc ex.c
ex.c: In function 'main':
ex.c:3:22: error: invalid type argument of unary '*' (have 'int')
   3 |     int result = *a;
     |                   ^
ex.c:6:18: error: invalid type argument of unary '*' (have 'int')
   6 |     result = *b;
     |              ^
```

[설명]

pointer 타입이 아닌 int 타입의 변수가 STAR 연산을 해서 error message가 출력된다. 제작한 컴파일러에서는 동일한 결과가 두 번 나와야 정상인데 한 번만 출력되고 segmentation

fault가 나는 것을 확인할 수 있다. gcc는 두 번 모두 동일하게 포인터 타입이 아닌 int 타입 이어서 error message를 출력하는 것을 확인할 수 있다.

10.

[입력]

```
float f(int);

int main(int a, char c[], float (*f)(int),...){

}
```

[출력]

```
| | | | | (ID="f") TYPE:9e83fc40 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=20
| | | | | TYPE
| | | | | POINTER
| | | | | ELEMENT_TYPE
| | | | | FUNCTION
| | | | | PARAMETER
| | | | | | (ID="") TYPE:9e83a2f0 KIND:PARAM SPEC=NULL LEV=2 VAL=0 ADDR=12
| | | | | | TYPE
| | | | | | | (int)
| | | | | | TYPE
| | | | | | (float)
| | | | | (ID="") TYPE:0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | TYPE
| | | | | (int)
| | | | | BODY
| | | | | N_STMT_COMPOUND (0,0)
| | | | | N_STMT_LIST_NIL (0,0)
| | | | | :-$ gcc ex.c
```

[설명]

함수의 파라미터들은 각각의 주소를 부여받는데, 배열 타입의 c는 파라미터 부분에서 포인터 타입이기 때문에 크기가 4가 되어 다음 f가 'c'의 주소+4가 되는 것을 확인할 수 있다. 또한, 함수 포인터 역시도 크기가 4이다. 해당 함수 내부의 파라미터들은 12부터 시작하는 주소를 받아 주소를 계산하게 된다. 'DOTDOTDOT'은 주소를 부여받지 않고, 타입은 int 타입이다. gcc 역시 제대로 출력된다.

11.

[입력]

```
int main(){
    int a[3];
    sizeof(a[2]);
    sizeof(a[0]);
    sizeof(a);
    sizeof(main);
    printf("%d",sizeof(void));
}
```

[출력]

```
**** semantic error at line 6: illegal type size in sizeof operation
**** semantic error at line 7: illegal type size in sizeof operation
: $ gcc ex.c
```

[설명]

선언한 배열의 크기가 0이 아니면 배열 타입을 sizeof 연산에 사용할 수 있다. 배열 전체도 가능하다. 하지만 함수 타입과 void 타입은 가능하지 않아 error message가 출력된다. gcc에서는 모두 가능하다고 한다. void의 sizeof는 1이다.

12.

[입력]

```
int main(){
    int a;
    int* b;
    int c[3];
    for( ; a ; );
    for(;;);
    for( ; b ; );
    for( ; c ; );
}
```

[출력]

```
**** semantic error at line 8: scalar type expr required in middle of for-expr
:~$ gcc ex.c
```

[설명]

for문 중간은 비어있거나 scalar type의 수식이 사용될 수 있다. 따라서, 배열 타입의 c는 error message가 출력된다. 반면 gcc는 모두 허용하고 있다.

13.

[입력]

```

int main(){
    switch(5/2){
        int a;
        struct { int b; } x;
        case 2:
            break;
        case a:
            continue;
        case x:
        default:
            return 0;
    }
}

```

[출력]

```

**** semantic error at line 7: illegal identifier a in constant expression
**** semantic error at line 8: illegal expression type in case label
**** semantic error at line 8: continue statement not within a loop
**** semantic error at line 9: illegal identifier x in constant expression
**** semantic error at line 11: illegal expression type in case label

```

```

$ gcc ex.c

```

```

ex.c: In function 'main':

```

```

ex.c:7:17: error: case label does not reduce to an integer constant

```

```

 7 |             case a:
   |             ^~~~~

```

```

ex.c:8:25: error: continue statement not within a loop

```

```

 8 |             continue;
   |             ^~~~~~

```

```

ex.c:9:17: error: case label does not reduce to an integer constant

```

```

 9 |             case x:
   |             ^~~~~

```

[설명]

case_statement에서는 정수 상수 수식만 가능하기 때문에 int 타입 변수, 구조체 타입 변수는 안되고, '2'는 가능하다. 또한, switch 안에서는 continue 사용이 불가하다. gcc 역시 동일한 부분을 error message로 출력하고 있다.

14.

[입력]

```
int main(){
    switch(5/2){
        int a;
        char b[2];
        struct { int b; } x;
        case 2:
            return 23.5;
        case 3:
            return b;
        case 4:
        default:
            return 0;
    }
}
```

[출력]

```
... warning at line 7: incompatible types in argument or return expr
**** semantic error at line 9: not permitted type conversion in return expression
$ gcc ex.c
ex.c: In function 'main':
ex.c:9:32: warning: returning 'char *' from a function with return type 'int' makes integer from pointer without a cast [-Wint-conversion]
     9 |         return b;
       |         ^
```

[설명]

main의 리턴 타입이 int(t1)이고, '23.5'는 float 타입(t2)으로 호환되지 않아 warning message가 출력된다. 배열 타입은 리턴 타입이 될 수 없어 error message가 출력되고 정수 상수인 0은 정상으로 판단된다. gcc에서는 배열 타입만 warning message로 출력하고 있다.

15.

[입력]

```
int f();

int main(){
    int arr[3];
    arr=0;
    3++;
}
```

[출력]

```
**** semantic error at line 5: expression is not an lvalue
**** semantic error at line 5: not permitted type casting in expression
**** semantic error at line 6: expression is not an lvalue
$ gcc ex.c
ex.c: In function 'main':
ex.c:5:12: error: assignment to expression with array type
   5 |         arr=0;
     |         ^
ex.c:6:10: error: lvalue required as increment operand
   6 |         3++;
     |         ^
```

[설명]

치환 연산자는 왼쪽 자식이 lvalue여야하는 배열 포인터는 lvalue가 아니어서 error message가 나온다. 형변환도 될 수 없다. 또한, 후위 증가 연산자는 lvalue가 operand로 있어야하는데 3은 정수 상수라 마찬가지로 error다. gcc에서도 동일한 결과를 보인다.

16.

[입력]

```
int main(){
    for(;;){
        case 3: break;
        while(1){
            default: continue;
        }
    }
}
```

[출력]

```
**** semantic error at line 3: case label not within a switch statement
**** semantic error at line 5: default label not within a switch statement
$ gcc ex.c
ex.c: In function 'main':
ex.c:3:17: error: case label not within a switch statement
   3 |         case 3: break;
     |         ^~~~~~
ex.c:5:25: error: 'default' label not within a switch statement
   5 |             default: continue;
     |             ^~~~~~
```

[설명]

for와 while에서는 case와 default가 나올 수 없다. 따라서 error message가 출력되며, continue와 break는 가능하다. gcc도 같은 결과를 보여준다.

17.

[입력]

```
int main(){
    do{
        case 0: continue;
        case 1: break;
        case 2:
        default:
            switch(2/1){
                case 3: break;
            }
    }while(1);
}
```

[출력]

```
**** semantic error at line 3: case label not within a swltch statement
**** semantic error at line 4: case label not within a switch statement
**** semantic error at line 9: case label not within a switch statement
**** semantic error at line 9: default label not within a switch statement
: $ gcc ex.c
ex.c: In function 'main':
ex.c:3:17: error: case label not within a switch statement
   3 |         case 0: continue;
     |         ^~~~~~
ex.c:4:17: error: case label not within a switch statement
   4 |         case 1: break;
     |         ^~~~~~
ex.c:5:17: error: case label not within a switch statement
   5 |         case 2:
     |         ^~~~~~
ex.c:6:17: error: 'default' label not within a switch statement
   6 |         default:
     |         ^~~~~~
```

[설명]

do-while에서는 case와 default는 불가능하고, continue와 break는 가능하다. 내부에 있는 case_statement는 case와 default, break를 사용할 수 있고 부모로부터 권한을 받아 continue 사용 가능하다.

18.

[입력]

```
int main(){
    float a[-2];
    float b[0-2];
    char c[0];
    char d['a'];
}
```

[출력]

```

**** semantic error at line 2: illegal array size or type
**** semantic error at line 2: illegal array size or empty array
**** semantic error at line 3: illegal array size or type
**** semantic error at line 3: illegal array size or empty array
**** semantic error at line 4: illegal array size or type
**** semantic error at line 4: illegal array size or empty array
**** semantic error at line 5: illegal array size or type
**** semantic error at line 5: illegal array size or empty array

```

```

: $ gcc ex.c

```

```

ex.c: In function 'main':
ex.c:2:15: error: size of array 'a' is negative
    2 |         float a[-2];
      |               ^
ex.c:3:15: error: size of array 'b' is negative
    3 |         float b[0-2];
      |               ^

```

[설명]

배열의 크기는 정수 상수여야하고, 0 이하는 불가능하다. 또한, 문자도 불가능하다. gcc에서는 배열의 크기로 0과 문자를 허용한다.

19.

[입력]

```

int main(){
    void arr[10];
    int arr2[2];
    float ar3[3];
    char ar[3];
    int (*arrrr[5])();
    int arrrrr[3](void);
}

```

[출력]

```

:~$ ./compiler < ex.c
line 7: syntax error: illegal type specifier in formal parameter near )
:~$ gcc ex.c
ex.c: In function 'main':
ex.c:2:14: error: declaration of 'arr' as array of voids
    2 |         void arr[10];
      |               ^~~~~
ex.c:7:13: error: declaration of 'arrrrr' as array of functions
    7 |         int arrrrr[3](void);
      |               ^~~~~~

```

[설명]

배열은 매개변수를 가질 수 없다. 따라서 error message가 출력된다. gcc에서는 arr가 void 타입으로 선언돼 error를 출력한다. 그리고 제작한 컴파일러와 마찬가지로 배열은 매개변수를 가질 수 없다.

20.

[입력]

```
struct { void a; int b; };
```

[출력]

```
**** semantic error at line 1: illegal type in struct or union field
:~$ gcc ex.c
ex.c:1:15: error: variable or field 'a' declared void
  1 | struct { void a; int b; };
    |             ^
ex.c:1:8: warning: unnamed struct/union that defines no instances
  1 | struct { void a; int b; };
    |      ^
```

[설명]

void 타입의 field는 불가능해서 error message를 출력한다. gcc에서도 동일한 결과를 보인다. 그리고 instance 이름이 없는 구조체임을 경고한다.

21.

[입력]

```
union { void a; int b; };
```

[출력]

```
**** semantic error at line 1: illegal type in struct or union field
:~$ gcc ex.c
ex.c:1:14: error: variable or field 'a' declared void
  1 | union { void a; int b; };
    |             ^
ex.c:1:7: warning: unnamed struct/union that defines no instances
  1 | union { void a; int b; };
    |      ^
```

[설명]

이전(20번)과 동일하다. void 타입의 field는 불가능하다. gcc에서는 instance의 이름이 없는 union임을 경고한다.

22.

[입력]

```
union { char a[3]; int f(); };
```

[출력]

```
**** semantic error at line 1: illegal type in struct or union field
:~$ gcc ex.c
ex.c:1:24: error: field 'f' declared as a function
  1 | union { char a[3]; int f(); };
    |                      ^
ex.c:1:7: warning: unnamed struct/union that defines no instances
  1 | union { char a[3]; int f(); };
    |      ^
```

[설명]

함수 타입은 union의 field의 타입이 될 수 없다. gcc도 동일한 결과를 보인다.

23.

[입력]

```
ex.c:1:22: error: enumerator value for 'x' is not an integer constant
  1 | enum { a, t = 3, x = 2.5 };
    |                      ~~~~~
```

[출력]

```
**** semantic error at line 1: integer type expression required in enumerator
:~$ gcc ex.c
ex.c:1:22: error: enumerator value for 'x' is not an integer constant
  1 | enum { a, t = 3, x = 2.5 };
    |                      ~~~~~
```

[설명]

enum에서는 정수 상수만 가능하다. 따라서 수식에 실수는 불가하다. gcc도 동일한 결과를 보인다.

24.

[입력]

```
int main(){
    int b = 234 % 3;
    int c = 234 % b;
    int d = 234 % 'c';
    int e = 234 % 3.2;
}
```

[출력]

```
세그멘테이션 오류 (코어 덤프됨)
:~$ gcc ex.c
ex.c: In function 'main':
ex.c:5:21: error: invalid operands to binary % (have 'int' and 'double')
    5 |         int e = 234 % 3.2;
      |                      ^

```

[설명]

'%' 이항연산자는 왼쪽 수식, 오른쪽 수식 둘 다 서수 타입이어야 한다. 따라서 실수 타입이나 다른 함수, 구조체, 배열은 불가능하다.

25.

[입력]

```
int main(){
    int b = 123 == 566;
    int c = (float)123 != 'a';
    enum { a }x;
    c == a;
}
```

[출력]

```

| | | | | | | | | | (ID="a") TYPE:0 KIND:ENUM_LITERAL SPEC=NULL LEV=1
VAL=0 ADDR=0
| | | | | | | | | | N_STMT_LIST (0,0)
| | | | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | | N_EXP_EQL (279092f0,0)
| | | | | | | | | | N_EXP_IDENT (279092f0,1)
| | | | | | | | | | (ID="c") TYPE:279092f0 KIND:VAR SPEC=AUTO LEV=1 VA
L=0 ADDR=16
| | | | | | | | | | N_EXP_IDENT (279092f0,0)
| | | | | | | | | | (ID="a") TYPE:0 KIND:ENUM_LITERAL SPEC=NULL LEV=1
VAL=0 ADDR=0
| | | | | | | | | | N_STMT_LIST_NIL (0,0)
:~$ gcc ex.c

```

[설명]

관계 연산자는 (산술형, 산술형)이 가능하므로 문제 없이 제작한 컴파일러, gcc가 정상적으로 종료된다.

26.

[입력]

```
int main(){
    int* a;
    int* b;
    float* c;
    a==b;
    a==c;
}
```

[출력]

```
**** semantic error at line 6: illegal expression type in relational operation
:~$ gcc ex.c
ex.c: In function 'main':
ex.c:6:10: warning: comparison of distinct pointer types lacks a cast
    6 |     a==c;
      |         ^
```

[설명]

관계 연산자는 호환적인 두 포인터 연산이 가능하다. 따라서 같은 int 포인터 타입은 호환적이고 int 포인터와 float 포인터는 호환적이지 않다. 제작한 컴파일러, gcc 모두 동일하게 결과를 출력한다.

27.

[입력]

```
int main(){
    int a;
    enum { b , c };
    333 < 555;
    a <= 5;
    b > 'c';
    c >= 3;
}
```

[출력]

```
| | | | | | | | | | N_EXP_INT_CONST (5b49f2f0,0)
| | | | | | | | | | INT=3
| | | | | | | | | | N_STMT_LIST_NIL (0,0)
:~$ gcc ex.c
```

[설명]

모두 산술형이어서 error 없이 정상적으로 종료된다.

28.

[입력]

```
int main(){
    typedef int INT;
    sizeof(INT);
    sizeof(float);
    sizeof(main);
    sizeof(1=2=3);
}
```

[출력]

```
**** semantic error at line 5: illegal type size in sizeof operation
**** semantic error at line 6: expression is not an lvalue
**** semantic error at line 6: expression is not an lvalue
:~$ gcc ex.c
ex.c: In function 'main':
ex.c:6:19: error: lvalue required as left operand of assignment
   6 |         sizeof(1=2=3);
     |                   ^

```

[설명]

sizeof는 type과 expression이 가능하다. main이 함수 포인터여서 semantic error message가 출력되고, 치환 연산자는 왼쪽 자식으로 lvalue여서 오류가 난다. gcc도 마찬가지로 lvalue에 대한 error message를 보내지만 main 함수 포인터가 sizeof에 있는 것은 정상 처리된다.

29.

[입력]

```
int main(){
    !1;
    float a;
    char b;
    !a;
    !b;
    !'c';
    !main;
}
```

[출력]

```
:~$ ./compiler < ex.c
line 3: syntax error near float
:~$ gcc ex.c

```

[설명]

main 함수 첫줄에 연산을 하고 syntax error가 나긴하지만 연산에는 문제 없다. gcc 역시도 정상 종료된다.

30.

[입력]

```
int main(){
    int* a;
    float b;
    &(--a);
    &b;
}
```

[출력]

```
**** semantic error at line 4: expression is not an lvalue
:~$ gcc ex.c
ex.c: In function 'main':
ex.c:4:9: error: lvalue required as unary '&' operand
    4 |         &(--a);
      |         ^
```

[설명]

AMP 연산은 lvalue를 요구하는데, --a는 lvalue가 아니다. a는 lvalue지만, PRE_DEC 연산에 의해 lvalue가 아니게 된다. 제작한 컴파일러, gcc 모두 동일하게 결과를 내고 있고, &b에서 b는 lvalue이기 때문에 정상 처리된다.

31.

[입력]

```
int main(){
    int e[2][3];
    int *p = e[0];
    p+=1;
}
```

[출력]

```
:~$ ./compiler < ex.c
line 4: syntax error near =
:~$ gcc ex.c
```

제작한 컴파일러에서 토큰을 잘못 자르는 것인지 syntax error가 난다. 따라서 제작한 컴파일러에서의 결과를 확인하기 어렵지만 gcc에 의하면 정상 처리된다.